Governed Shared Memory for Cross-Team AI Agents: Permission-Aware Hybrid Retrieval with Versioning and Auditability

[Student Name]*
Department of Computer Science and Engineering, [Institute Name]
[student-email]

[Guide Name]
Department of Computer Science and Engineering, [Institute Name]
[guide-email]

September 2026

Abstract

Large-language-model (LLM) agents are increasingly deployed per team—for development, quality assurance, security and customer support—but each agent keeps a private memory. The result is repeated investigation, duplicated reasoning cost, knowledge silos and contradictory records. Pooling all agent knowledge into a single vector database is not an acceptable fix, because semantic similarity is not authorisation: a support agent's question can retrieve a confidential security finding. We present the *Shared Memory Layer* (SML), a service through which agents of different teams store, retrieve, share and version knowledge under explicit governance. SML expresses one access-control predicate in three enforcement forms—application code, SQL and a vector-store filter—so that unauthorised memories are never scored by the similarity search and every candidate is re-checked against the system of record. It combines dense and full-text retrieval under the same filter, keeps immutable versions and provenance, flags duplicates and contradictions for human review, and writes an append-only audit trail. Agents connect through a REST API, the Model Context Protocol and client SDKs. On an in-house cross-team benchmark, sharing raised Hit@1 from 0.35 to 0.975 and removed all duplicated work (62.5 % → 0 %), while 0 of 140 adversarial access attempts leaked. On three public benchmarks the permission-aware hybrid search outperformed BM25 and a plain vector store: Recall@10 of 0.673 on LoCoMo (the benchmark authors' DRAGON retriever: 0.675), 0.807 on LongMemEval and nDCG@10 of 0.718 / 0.359 on BEIR SciFact / NFCorpus, with a 33M-parameter local embedding model and a median search latency of 83–109 ms at up to 102,229 memories.

Keywords: multi-agent systems; agent memory; retrieval-augmented generation; access control; vector search; hybrid retrieval; auditability.

1 Introduction

AI agents now write code, test releases, review security findings and answer customers. In most deployments each agent is configured per team and keeps its own memory, whether that memory is a chat history, a scratch-pad file or a private vector index. When the Development agent learns that authentication fails because refresh tokens expire, the QA agent that meets the same symptom a week later cannot benefit: it re-investigates, spends the same LLM reasoning budget

*Replace the bracketed placeholders with the author details before submission.

again, and may record a slightly different—or contradictory—conclusion. The organisation pays repeatedly for knowledge it already has.

The obvious remedy, one shared vector database, introduces a security problem that is easy to miss. Semantic search ranks every stored item by similarity to the query. If confidential memories are part of the index, a harmless-looking question from an unauthorised agent can retrieve them, and any filter applied *after* ranking is a single point of failure. Organisations also need what a plain index does not offer: who created a piece of knowledge, how it changed, whether two agents disagree, and who read or was refused access.

This paper describes and evaluates the Shared Memory Layer (SML), a governed memory service for agents of different teams. Its central idea is that authorisation must be part of retrieval, not a step after it. Our contributions are:

1. **A single access predicate in three enforcement forms.** One read rule over five visibility levels is implemented as application logic, as a SQL predicate and as a vector-store payload filter. Similarity search therefore never scores an unreadable memory, and every result is re-checked against the relational system of record (Section 4.2).
2. **Permission-aware hybrid retrieval.** Dense nearest-neighbour search and PostgreSQL full-text search run under the identical filter and are merged with a transparent scoring function (Section 4.3).
3. **Knowledge governance.** Immutable versions, provenance, semantic duplicate detection, rule-based contradiction flags with human review, and an append-only audit log that records denials (Section 4.4).
4. **A complete, open implementation** with REST, Model Context Protocol (MCP) and SDK interfaces, a dashboard and one-command deployment (Section 5).
5. **An evaluation** on an in-house cross-team benchmark, an adversarial security suite and three public benchmarks—LoCoMo, LongMemEval and BEIR—against BM25, a plain vector store and published results of other memory layers (Section 6).

2 Related Work

Multi-agent LLM frameworks. AutoGen [1], MetaGPT [2] and CAMEL [3] coordinate several LLM agents through conversation or a shared message pool. Shared state lives for one run or one group chat; there is no persistent, access-controlled memory across teams. Conceptually SML is a governed *blackboard* [4]: agents coordinate indirectly through persistent knowledge instead of point-to-point messages.

Memory for LLM agents. Generative Agents [5] retrieve from a memory stream by recency, importance and relevance; MemGPT [6] pages information between a context window and external storage. Production memory layers such as Mem0 [7], Zep [8], LangMem [9] and A-Mem [10] extract facts or build (temporal) knowledge graphs with an LLM at ingestion time and report answer accuracy on long-conversation benchmarks. These systems optimise memory for one user or one agent; scoping is by user, agent or project identifier. None of the published evaluations addresses per-memory authorisation across teams, leakage under similarity attacks or auditability, which are the focus of this work.

Retrieval. Retrieval-augmented generation [11] conditions an LLM on retrieved text. Dense retrievers [12–14] and general-purpose embedding models such as BGE [15] and MiniLM [16] capture meaning beyond shared words; lexical ranking with BM25 [17] remains a strong baseline for exact terms, and late-interaction models such as ColBERT [18] trade cost for quality. BEIR [19] and MTEB [20] are the standard benchmarks for these components. Approximate

nearest neighbour search with HNSW graphs [21] underlies vector databases such as Qdrant [22], which can evaluate payload filters inside the index.

Access control. Role-based access control (RBAC) [23] governs *operations*; attribute-based access control (ABAC) [24] evaluates attributes of subjects and objects. SML combines RBAC for operations with attribute-based, per-object visibility for memories.

Table 1 summarises the gap.

Table 1: Capabilities of related approaches (✓ supported, ∼ partial / session-scoped / via extensions, – not addressed), assessed from the designs described in the cited papers and documentation.

Approach	Persistent memory	Cross-team sharing	Semantic retrieval	Per-memory access control	Versions & provenance	Security audit trail
AutoGen, MetaGPT, CAMEL	∼	∼	∼	–	–	–
Generative Agents, MemGPT	✓	–	✓	–	–	–
RAG over a vector store	✓	∼	✓	∼	–	–
Mem0	✓	∼	✓	∼	∼	–
Zep	✓	∼	✓	–	∼	–
SML (this work)	✓	✓	✓	✓	✓	✓

3 Problem Formulation and Threat Model

Entities. An organisation has teams $\mathcal{T}$. A *principal* p is either an agent (with its own API key) or a human user; it belongs to at most one team $\tau(p)$, holds one or more roles and thereby a set of permissions (e.g. `memory:create`, `memory:read`, `memory:share`). A *memory* m has a title, content, type, tags, importance and confidence in $[0, 1]$, an owning team $\tau(m)$, a creator $c(m)$, a visibility $v(m)$, a sensitivity level (public, internal, confidential, restricted), an optional expiry time, explicit sharing grants to teams $G_T(m)$ and agents $G_A(m)$, and a version history.

Visibility. Table 2 lists the five visibility levels. Confidential and restricted memories may not be organisation-wide.

Table 2: Memory visibility levels.

Visibility	Readable by (besides the creator and administrators)
`private`	nobody else; private memories cannot be shared
`team`	members of the owning team, plus explicit team or agent grants
`selected_teams`	the owning team plus the explicitly allowed teams
`selected_agents`	only the explicitly allowed agents
`organization`	every authenticated principal

Read predicate. For a non-administrator principal p holding `memory:read`, memory m is readable, $R(p, m)$, if and only if m is active, not expired (unless expired memories are explicitly

requested), and

$$
\begin{aligned}
& c(m) = p \ \vee \ v(m) = \texttt{organization} \\
& \vee \ \Big(v(m) \neq \texttt{private} \ \wedge \ \big[(\tau(p) = \tau(m) \wedge v(m) \in \{\texttt{team}, \texttt{selected_teams}\}) \\
& \qquad \vee \ \tau(p) \in G_T(m) \ \vee \ p \in G_A(m) \big] \Big).
\end{aligned} \tag{1}
$$

Archived memories are visible to administrators only. Modification (update, archive, share) additionally requires the creator or a manager of the owning team.

Goals. (G1) An agent can retrieve, by meaning, every memory it is authorised to read, including other teams' knowledge. (G2) No operation—read, search, update, version history, sharing—discloses a memory for which $R(p, m)$ is false, including through similarity ranking, duplicate warnings or conflict notices. (G3) Every change is versioned and attributable. (G4) Every access and every refusal is recorded in a tamper-resistant log.

Adversary. We consider authenticated but unauthorised principals: agents of other teams who issue arbitrary requests, including semantic searches that use the protected memory's own title or content as the query (the strongest similarity attack), attempts to share a memory to themselves, and privilege escalation (creating teams or agents, reading the audit log, listing users, re-indexing). Token theft is mitigated by short-lived tokens and refresh-token rotation; compromise of the database or of the host is out of scope.

4 System Design

4.1 Architecture

Figure 1 shows the components. Agents and human users call a single API gateway. Every request is authenticated (a JWT [25] or an agent API key) and authorised twice: RBAC decides whether the principal may perform the operation, and the read predicate (1) decides whether it may touch the specific memory. PostgreSQL is the system of record for memories, versions, grants, identities and the audit log. Qdrant stores one vector per memory together with the payload fields needed to evaluate the read predicate. Embeddings are produced locally by `bge-small-en-v1.5` [15] (384 dimensions, 33M parameters), so no text leaves the deployment.

4.2 One access rule, three enforcement forms

A naive design ranks all memories by similarity and filters the results afterwards. Two failure modes follow: unreadable memories still participate in ranking (and can surface in any code path that forgets the filter), and a single bug in the post-filter exposes everything. SML instead implements the read predicate (1) three times and keeps the forms synchronised by tests:

1. `can_read` evaluates a concrete PostgreSQL row and is authoritative;
2. `sql_read_filter` is a SQL predicate used for listings and full-text search;
3. `build_read_filter` is a Qdrant payload filter (a disjunction of conjunctions over visibility, owning team, creator, grants, status and expiry) that is pushed *into* the approximate nearest-neighbour query.

Because the filter is part of the vector query, an unreadable memory is never a candidate, however similar it is. Every candidate is nevertheless re-checked with `can_read` against the current row before it is returned (defence in depth, and protection against a stale vector payload). Duplicate warnings are computed only among readable memories, and conflicts with unreadable memories are reported to administrators while the caller only learns their count.

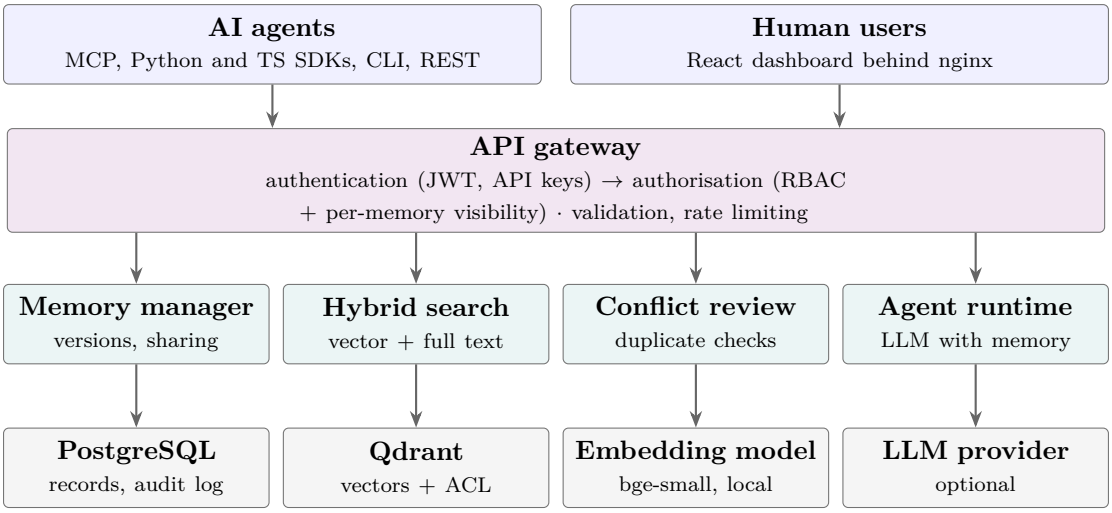


Figure 1: Architecture of the Shared Memory Layer. Every path to a memory passes through authentication and two-level authorisation; audit records are written in an independent transaction.

4.3 Permission-aware hybrid retrieval

Embeddings capture paraphrase but blur exact identifiers such as error codes, CVE numbers or ticket ids. SML therefore runs two candidate generators under the same filter (Algorithm 1): a dense search in Qdrant and a PostgreSQL full-text search [26] over a generated, weighted `tsvector` (title weight A, content weight B) with `websearch_to_tsquery`, in which every query term must match. Keyword-only candidates receive their true cosine similarity from Qdrant, so all candidates are scored on the same scale:

$$s(m) = 0.90\,\mathrm{sim}(q,m) + 0.05\,\mathrm{imp}(m) + 0.05\,\mathrm{conf}(m) + 0.02\,[\mathrm{verified}(m)] + 0.10\,\mathrm{kw}(q,m), \quad (2)$$

where kw is the `ts_rank_cd` rank normalised by the best keyword hit (0 for memories that do not contain every query term). The weights favour meaning, let human-verified and high-confidence knowledge win ties, and give exact matches a bonus. If the vector store is unavailable, search degrades to permission-filtered keyword search instead of failing.

Algorithm 1 Permission-aware hybrid search

Require: principal p, query q, metadata filters F, result size k
$\phi \leftarrow \textsc{ReadFilter}(p) \wedge F$ ▷ visibility, grants, status, expiry, type, project, ...
$C_{\mathrm{kw}} \leftarrow$ full-text matches of q in PostgreSQL under ϕ (all terms), ranked by `ts_rank_cd`
$\mathbf{v} \leftarrow \textsc{Embed}(q)$
$C_{\mathrm{vec}} \leftarrow$ the $3k$ nearest neighbours of $\mathbf{v}$ in Qdrant under ϕ
$C \leftarrow C_{\mathrm{kw}} \cup C_{\mathrm{vec}}$; obtain cosine similarity for keyword-only candidates
for all $m \in C$ **do**
  if not $\textsc{CanRead}(p, m)$ on the current PostgreSQL row **then**
    remove m from C ▷ defence in depth
  end if
  compute $s(m)$ with Equation (2)
end for
update access and cross-team reuse counters; append a `MEMORY_SEARCH` audit record
return the k highest-scoring memories of C

4.4 Write path, lifecycle and governance

Creation. A new memory is validated (lengths, tag format, no markup in titles, sensitivity rules), authorised (`memory:create`; the body may not name another agent or team), embedded, and checked for duplicates: an identical content hash or a readable memory with cosine similarity ≥ 0.92 yields a warning (or a rejection, if requested). Organisation-wide candidates with similarity ≥ 0.70 are passed to contradiction rules (e.g. differing numeric values or opposite technology choices for the same subject), which open a conflict for human review. The row, version 1 and grants are flushed to PostgreSQL, the vector is upserted into Qdrant, and the transaction commits; if the commit fails the vector is deleted, and an administrator endpoint reports and repairs any divergence by re-embedding from PostgreSQL.

Versioning and provenance. Updates lock the row, check an optional expected version (optimistic concurrency), increment the version and store an immutable snapshot linked to its predecessor. There is exactly one vector per memory, so search always returns the latest version while older versions remain readable to authorised principals. Provenance reports the creator, source, modifiers and access history. Humans resolve conflicts with *keep new*, *keep existing*, *keep both* or *dismiss*; nothing is silently overwritten.

Auditability. Logins, token events, every memory operation, sharing changes, role and permission changes, conflict resolutions and every denial are recorded with actor, team, *on-behalf-of* principal, resource, status, reason and request identifier. Audit writes use an independent transaction, so refused operations are always recorded, and a database trigger rejects `UPDATE` and `DELETE` on the audit table.

Identity security. Passwords are hashed with Argon2id [27]. Access tokens are HS256 JWTs valid for 15 minutes; opaque refresh tokens rotate on every use, and re-use of a rotated token revokes its whole family. Agent API keys are stored only as keyed hashes. Authentication endpoints are rate limited per client and repeated failures lock an account temporarily.

4.5 Agent interfaces

Agents use SML through (i) a REST API documented with OpenAPI (57 operations), (ii) a Model Context Protocol [28] endpoint exposing eight tools—`search_memory`, `remember`, `get_memory`, `update_memory`, `share_memory`, `memory_history`, `list_teams`, `whoami`—so that off-the-shelf MCP clients can use the memory without code, and (iii) Python and TypeScript SDKs with a command-line tool, bulk import of Markdown, JSON and CSV knowledge, and ready-made tool definitions for LLM function calling. All interfaces execute as the calling agent through the same services, so authorisation, rate limits and auditing are identical. A built-in agent runtime retrieves memories for a task, passes them to a configurable LLM (local or hosted) and can store the result as a new memory.

5 Implementation

SML is implemented in Python 3.12 with FastAPI and Pydantic v2 (API), SQLAlchemy 2 and Alembic (PostgreSQL 16 schema with foreign keys, check constraints, indexes, a generated full-text column and the append-only audit trigger), Qdrant 1.13 (HNSW with payload indexes on every filter field) and fastembed/ONNX for local embeddings. A React/TypeScript dashboard provides monitoring, a memory explorer, access-control management, the audit log, a conflict-review queue and an *agent playground* in which a human simulates an agent under that agent's permissions (audited on the human's behalf). The whole system runs as four containers with

one `docker compose` command. The test suite comprises 105 backend tests (unit, security and integration tests against real PostgreSQL, Qdrant and the embedding model, including an end-to-end MCP test with the official client), 14 Python SDK tests, 5 TypeScript SDK tests and a 16-step acceptance scenario against the running deployment; all pass.

6 Evaluation

We answer four questions. **RQ1:** does shared memory improve retrieval and reduce duplicated work compared with per-agent memory? **RQ2:** does the access model prevent leakage, including through similarity? **RQ3:** how good is the retrieval on public benchmarks compared with standard baselines and published systems? **RQ4:** what latency does governed retrieval cost?

6.1 Setup

All experiments ran on one laptop (Windows 11 on ARM64, Docker Desktop with 12 virtual CPUs and 15.5 GB of memory; no GPU). Every memory was created and every question asked through the public API as an ordinary, permission-checked request. The public benchmarks ran against a second API process from the same image with its own database and collection (rate limiting disabled on that process only), so the 100,000+ benchmark memories did not affect the demonstration data or its audit log. Two baselines ran locally over exactly the same texts: **BM25** [17] (k_1=1.5, b=0.75) and a **plain vector store** using `all-MiniLM-L6-v2` [12, 16] with cosine similarity, the default embedding model of Chroma-style memories. We report SML in *hybrid* (default) and *semantic* (dense only) mode. Published results of other systems are quoted from their papers and were not re-run.

Validity of the scoring. Our local `all-MiniLM-L6-v2` baseline reproduces its published BEIR scores exactly (nDCG@10 0.645 on SciFact and 0.316 on NFCorpus, as in the MTEB results), and SML's semantic mode reproduces the published scores of `bge-small-en-v1.5` (0.717 vs. 0.713 and 0.342 vs. 0.343). The LongMemEval metrics were ported line by line from the official evaluation code and agree with it to 10^{-16} on 3,000 randomised rankings. Retrieval is deterministic: two independent LoCoMo runs produced identical recall.

6.2 RQ1: Shared versus isolated memory

Corpus. Four freshly created team agents (Development, QA, Security, Support) stored 35 labelled memories; 40 paraphrased questions (10 per team) have ground-truth relevant memories, and 25 of them can only be answered with another team's knowledge. *Isolated* restricts each agent to its own memories (the per-agent silo), *Shared* is SML, and *Lexical* ranks exactly the memories the agent may read with TF-IDF, separating the effect of embeddings from the effect of sharing.

Table 3 shows that sharing is what makes the cross-team questions answerable at all (Hit@5 0.375 → 1.000), while embeddings mainly improve *ranking* over keyword matching on the same authorised memories (Hit@1 0.850 → 0.975). In agent tasks, isolated agents had to re-derive 62.5 % of the answers; every one of those 25 re-stored memories was flagged by SML's duplicate detector. Without an LLM the shared agent completed tasks in 122 ms on average versus 265 ms for the isolated agent including re-derivation. With a small local LLM (`qwen2.5:0.5b` on CPU) the shared agent was about 20 % slower (4,283 vs. 3,574 ms) because it sends more retrieved context to the model, but every answer was grounded in existing organisational knowledge.

Table 3: In-house cross-team benchmark (35 memories, 40 queries, top-5).

Metric	Isolated (baseline)	**Shared (SML)**	Lexical (TF-IDF)
Hit@1	0.350	**0.975**	0.850
Hit@5	0.375	**1.000**	**1.000**
Recall@5	0.338	0.963	**0.988**
MRR	0.358	**0.983**	0.921
nDCG@5	0.338	**0.957**	0.927
Hit@5 on the 25 cross-team queries	0.000	**1.000**	**1.000**
Knowledge-reuse rate	0.000	**0.675**	–
Tasks resolved from existing memory	0.375	**1.000**	–
Duplicate-task rate (knowledge re-derived)	0.625	**0.000**	–
Re-derived memories flagged as duplicates	25 / 25	–	–

6.3 RQ2: Security

The adversarial suite let every agent outside the authorised teams attack each of seven protected memories through six paths, plus five privilege-escalation operations (Table 4).

Table 4: Adversarial security suite.

Attempt	Attempts	Blocked
Direct read of a protected memory	20	20
Update	20	20
Read version history	20	20
Share the memory to one's own team	20	20
Semantic search using the memory's title	20	20
Semantic search using the memory's full content	20	20
Privilege escalation (create team / agent, read audit log, list users, re-index)	20	20
Total	**140**	**140**

No attempt leaked, including the 40 searches that used the protected memory's own text as the query; the protected memories were never scored. All 100 expected denials were found in the audit log (the 40 search attempts are not denials and are audited as ordinary searches).

6.4 RQ3: Public benchmarks

Figure 2 summarises the headline metric of each benchmark; details follow.

LoCoMo. LoCoMo [29] contains very long multi-session conversations with questions whose evidence turns are annotated; it is the benchmark on which Mem0, Zep, LangMem and A-Mem report. We stored all 5,882 turns of the 10 released conversations (one project per conversation) and evaluated the 1,535 answerable questions (the adversarial category is excluded, as in the Mem0 evaluation). Recall@k is the share of evidence turns retrieved in the top k, the metric the LoCoMo authors report for their DRAGON retriever [14].

Hybrid retrieval is the best configuration at every cut-off (Table 5) and tracks DRAGON within 0.01 (Figure 3), although DRAGON has roughly three times as many parameters and was evaluated on the full 50-conversation set; we therefore read the result as *comparable*, not better. The margin over BM25 is largest on multi-hop and open-domain questions (+0.21 Recall@10, Table 6), which paraphrase their evidence. Requiring every query term (keyword-only mode) is precise but recalls evidence for only 12 % of questions, which is why SML uses full-text search

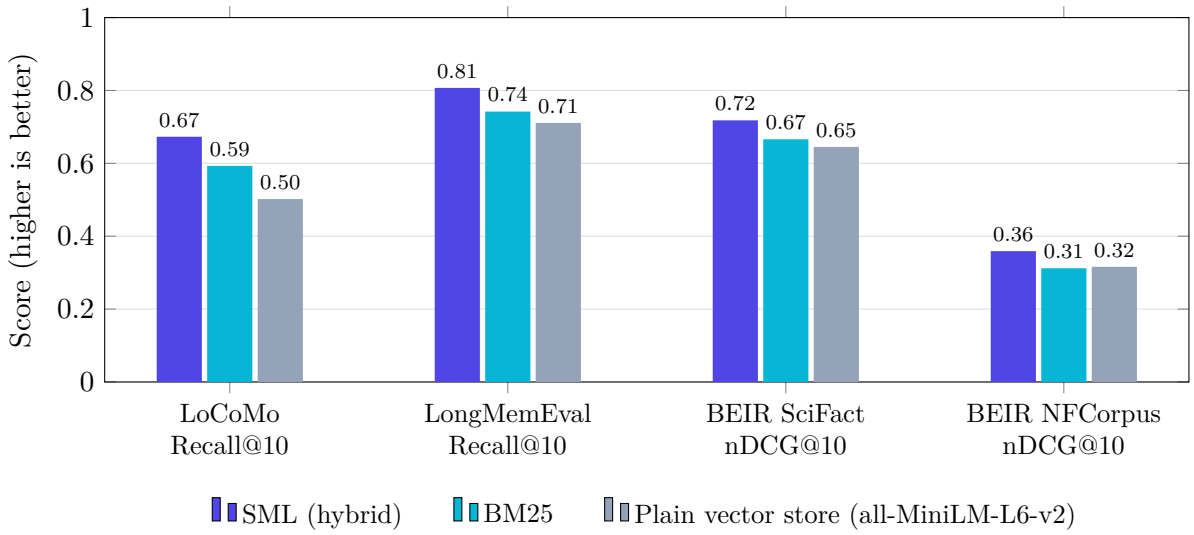


Figure 2: Headline metric of each public benchmark for SML and the two baselines run on identical data.

Table 5: LoCoMo evidence retrieval (1,535 questions). Bold: best measured value per column.

Configuration	R@5	R@10	R@25	R@50	Hit@1	MRR	nDCG@10
SML hybrid	**0.582**	**0.673**	**0.789**	**0.852**	**0.392**	**0.511**	**0.523**
SML semantic	0.561	0.657	0.783	0.848	0.352	0.478	0.494
SML keyword only (all terms)	0.119	0.121	0.122	0.123	0.118	0.126	0.118
BM25	0.523	0.593	0.669	0.719	0.345	0.450	0.463
Plain vector store	0.397	0.502	0.637	0.729	0.212	0.326	0.344
Recency (last k turns)	0.002	0.010	0.034	0.078	0.001	0.005	0.004
DRAGON (published, 50 conv.)	*0.588*	*0.675*	*0.799*	*0.848*	–	–	–

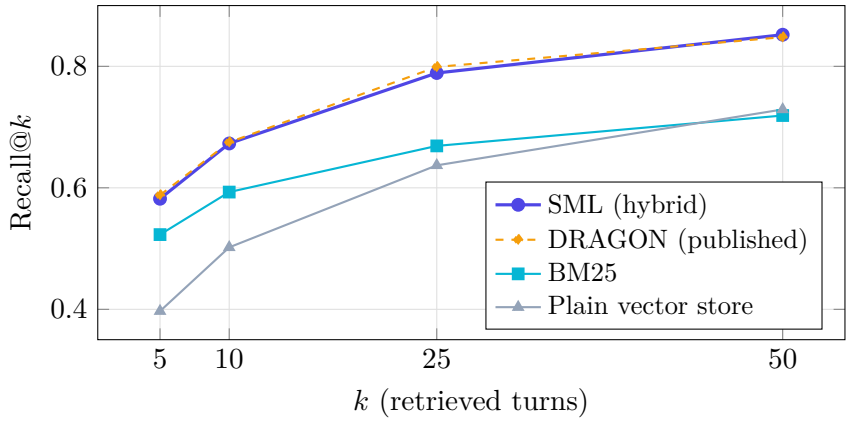


Figure 3: LoCoMo Recall@k: SML is within 0.01 of the benchmark authors' DRAGON retriever at every depth.

Table 6: LoCoMo Recall@10 by question category (number of questions).

Configuration	Single-hop (841)	Temporal (320)	Multi-hop (282)	Open-domain (92)
SML hybrid	**0.733**	**0.736**	**0.488**	**0.473**
SML semantic	0.711	0.720	0.485	**0.473**
BM25	0.699	0.688	0.277	0.267
Plain vector store	0.570	0.562	0.309	0.266

as a bonus rather than as the primary ranker. The top-10 context is about 441 tokens, 2 % of an average conversation (about 21,475 tokens).

LongMemEval. LongMemEval [30] evaluates the long-term memory of chat assistants: each question has its own history of about 48 sessions (about 115k tokens). We used LongMemEval$_S$ and followed the official retrieval protocol: every user message is a retrievable item, evidence messages are annotated, abstention questions and questions whose evidence lies only in assistant messages are skipped, Recall@k requires *all* evidence messages in the top k, and session-level metrics are derived from the message ranking. This yields 419 questions; storing each question's history in its own project produced 102,229 memories.

Table 7: LongMemEval$_S$ retrieval (419 questions, official protocol). The published Stella V5 1.5B results [30, 31] use LongMemEval$_M$, whose haystack is about ten times larger, and are a reference point only.

	Message level					Session level
Configuration	R@5	NDCG@5	R@10	NDCG@10	R@50	R@10
SML hybrid	**0.637**	0.663	**0.807**	**0.708**	**0.974**	0.907
SML semantic	**0.637**	0.656	0.802	0.700	**0.974**	**0.909**
BM25	0.628	**0.664**	0.742	0.693	0.840	0.783
Plain vector store	0.511	0.570	0.711	0.623	0.936	0.840
Stella V5 1.5B on M (published)	*0.582*	*0.481*	*0.692*	*0.512*	–	*0.783*

Table 8: LongMemEval$_S$ message-level Recall@10 by question type (number of questions).

Configuration	Multi-session (121)	Temporal (127)	Knowl. update (72)	User facts (64)	Preferences (30)	Assistant (5)
SML hybrid	**0.711**	0.732	**0.958**	0.953	**0.800**	**1.000**
SML semantic	**0.711**	0.724	**0.958**	0.938	**0.800**	**1.000**
BM25	0.562	**0.748**	0.931	**0.969**	0.500	0.800
Plain vector store	0.545	0.614	0.903	0.953	0.767	**1.000**

SML retrieves all evidence within the top 10 for 80.7 % of the questions and within the top 50 for 97.4 % (Table 7); at least one evidence message is in the top 10 for 93.8 %. The largest gains over BM25 are on multi-session and preference questions (Table 8), whose wording differs from the evidence. BM25 is marginally ahead on temporal and user-fact questions, which repeat exact names and dates, and ties SML on NDCG@5. Because the full-text component rewards only memories containing every query term, it rarely fires on long natural-language questions; a BM25-weighted lexical ranking fused with the dense ranking (e.g. reciprocal rank fusion [32]) is the natural next step.

BEIR. BEIR [19] measures zero-shot document retrieval, the component every memory layer depends on. We ingested SciFact (5,183 abstracts, 300 claims) and NFCorpus (3,633 medical documents, 323 questions with graded relevance and about 38 relevant documents per question) and report nDCG@10.

SML's hybrid search is above every open baseline in Table 9, including the two-stage BM25 plus cross-encoder pipeline, and within 0.016 (SciFact) and 0.024 (NFCorpus) of OpenAI's hosted `text-embedding-3-small`—while running a 33M-parameter model locally without an API key. On NFCorpus the full-text component adds 0.017 nDCG@10 over dense retrieval alone, reflecting the value of exact medical terms.

Table 9: BEIR nDCG@10. Upper block measured through the SML API; lower block published on the same test sets (BEIR paper [19] and MTEB results [20]).

Retriever	Source	SciFact	NFCorpus
SML hybrid (bge-small-en-v1.5, local)	measured	**0.718**	**0.359**
SML semantic	measured	0.717	0.342
BM25 (this evaluation)	measured	0.666	0.312
Plain vector store, all-MiniLM-L6-v2 (this evaluation)	measured	0.645	0.316
OpenAI text-embedding-3-small (Mem0's default embedder)	MTEB	0.734	0.383
OpenAI text-embedding-ada-002	MTEB	0.727	0.370
bge-small-en-v1.5	MTEB	0.713	0.343
BM25 + cross-encoder re-ranker	BEIR	0.688	0.350
ColBERT	BEIR	0.671	0.305
BM25 (Elasticsearch)	BEIR	0.665	0.325
all-MiniLM-L6-v2 (Chroma's default embedder)	MTEB	0.645	0.316
TAS-B	BEIR	0.643	0.319
ANCE	BEIR	0.507	0.237
DPR	BEIR	0.318	0.189

Comparison with published memory layers. Table 10 places SML next to the LoCoMo results reported in the Mem0 paper [7]. Those systems report *answer accuracy* judged by an LLM (GPT-4o-mini as answerer and judge) against hosted services; SML was evaluated on evidence retrieval, the input any answering model receives. The accuracy column is therefore left empty rather than estimated. On LongMemEval$_S$, Zep reports 71.2 % answer accuracy with GPT-4o (60.2 % for GPT-4o with the full history) [8]; SML places all evidence in the top 10 for 80.7 % of questions, a different but complementary measure.

Table 10: LoCoMo results of other memory layers as published in the Mem0 paper (Table 2), and SML as measured. Latencies of published systems include network access to hosted services.

System	Answer accuracy (LLM judge)	Search p50 / p95	Context tokens
Full context (no memory layer)	72.9 %	–	26,031
Mem0 with graph memory	68.4 %	476 / 657 ms	3,616
Mem0	66.9 %	148 / 200 ms	1,764
Zep	66.0 %	513 / 778 ms	3,911
LangMem	58.1 %	17,990 / 59,820 ms	127
A-Mem	48.4 %	668 / 1,485 ms	2,520
SML (hybrid)	not measured	**83 / 186 ms**	≈441 (top 10)

6.5 RQ4: Latency and scale

Table 11: Client-side search latency of SML's hybrid mode (top 50, milliseconds).

Workload	Memories in the collection	p50	p95
In-house benchmark (top 5)	35	103	208
LoCoMo	5,882	83	186
BEIR NFCorpus	3,633	90	180
BEIR SciFact (long abstracts)	5,183	109	197
LongMemEval$_S$	102,229	84	189

Table 11 shows that the median search latency stays between 83 and 109 ms from 35 to

102,229 memories, and p95 below 210 ms. Server-side time (embedding the query, the filtered vector search, the full-text query, the per-candidate re-check, ranking, access counters and the audit write) accounts for most of it; for example 62 ms at the median on LoCoMo. In a scale test from 100 to 1,000 memories, the server-side median grew only from 80 to 93 ms, indicating that per-request work, not the vector search, dominates. Creating a memory with duplicate and conflict detection took 189 ms at the median, and bulk import without duplicate detection reached 14–23 memories per second on the laptop CPU, bounded by embedding.

7 Discussion and Limitations

Retrieval quality is competitive, not state of the art in every row. SML matches the retriever used by the LoCoMo authors, beats BM25 and a plain vector store on every headline metric and every BEIR baseline except hosted OpenAI embeddings, but BM25 remains slightly better on questions dominated by exact names and dates. The hybrid bonus is deliberately conservative; learning or fusing the lexical signal is future work.

Answer accuracy was not measured. The published comparisons of memory layers grade LLM-written answers with an LLM judge. Retrieval recall bounds what any answering model can ground, but it is a different metric, and the two are never compared directly in this paper.

Published numbers were not reproduced. Figures of other systems come from different hardware, hosted services and, for DRAGON and Stella, larger data (50 LoCoMo conversations; LongMemEval$_M$). The baselines that were run (BM25, plain vector store) use exactly the same texts and questions.

In-house corpus. The 35-memory corpus and its paraphrased questions were written by the authors; absolute numbers from it will not transfer to large, noisy corpora, which is why the public benchmarks were added. The isolated baseline is an idealisation of per-agent memory, and the re-derivation cost is measured as one extra store, a lower bound for real re-work.

Single machine. All latencies were measured with co-located services on a laptop; they include container networking but not wide-area latency. Rate limiting was disabled on the benchmark process only.

No LLM-based consolidation. SML stores knowledge as written, which keeps provenance exact and ingestion cheap, but it does not summarise or consolidate memories; the LongMemEval authors report that LLM-extracted fact keys add about 0.09 Recall@10 on their larger setting.

8 Conclusion and Future Work

We presented the Shared Memory Layer, a governed memory service that lets AI agents of different teams reuse each other's knowledge without leaking what they must not see. Expressing one access predicate as application logic, SQL and a vector-store filter makes similarity search leak-free by construction and keeps a defence-in-depth re-check; versioning, duplicate and contradiction review and an append-only audit trail make shared knowledge accountable. Sharing raised Hit@1 on cross-team questions from 0.35 to 0.975 and removed duplicated work, no adversarial attempt leaked, and on three public benchmarks the permission-aware hybrid search outperformed BM25 and a plain vector store, matched the LoCoMo authors' retriever and came within 0.024 nDCG@10 of hosted OpenAI embeddings with a small local model.

Future work includes rank fusion with a BM25-weighted lexical signal and a cross-encoder re-ranker, LLM- or NLI-based contradiction detection, end-to-end answer-accuracy evaluation with an LLM judge, attribute-based policies expressed as code, horizontal scaling with a shared rate limiter, and a pilot with real teams' knowledge bases.

Reproducibility. The system, the benchmark scripts (in-house, LoCoMo, LongMemEval, BEIR), the raw results and the commands to reproduce every table are part of the project repository.

References

[1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[2] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[5] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[6] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[8] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[9] LangChain. LangMem: Long-term memory for LLM agents. `https://langchain-ai.github.io/langmem/`, 2025.

[10] Wujiang Xu et al. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[13] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[14] Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. How to train your DRAGON: Diverse augmentation towards generalizable dense retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[15] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[16] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[18] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.

[19] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[20] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023.

[21] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[22] Qdrant Team. Qdrant: Vector similarity search engine. `https://qdrant.tech/documentation/`, 2024.

[23] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[24] Vincent C. Hu, David Ferraiolo, Rick Kuhn, et al. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST SP 800-162, National Institute of Standards and Technology, 2014.

[25] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015.

[26] The PostgreSQL Global Development Group. PostgreSQL 16 documentation: Full text search. `https://www.postgresql.org/docs/16/textsearch.html`, 2023.

[27] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New generation of memory-hard functions for password hashing and other applications. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302, 2016.

[28] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024.

[29] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[30] Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

[31] Dun Zhang. Stella en 1.5B v5 text embedding model. `https://huggingface.co/dunzhang/stella_en_1.5B_v5`, 2024.

[32] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.